

Diagramatix — Quick Reminder Sheet 7

Domain Diagrams (UML Class)

A Domain diagram captures the *information* behind your processes using UML class-diagram notation: the entities that matter, what they hold, and how they relate. It is the conceptual data model that underpins systems, reports, and shared vocabulary. Use it for glossaries, data models, and to pin down the structure a BPMN process operates on. Diagramatix can also reproduce a domain diagram from an uploaded image (see Sheet 12).

ELEMENTS

Entity (class) — A box naming a thing in the domain, optionally with an **attributes** compartment (properties, with visibility, type, multiplicity) and an **operations** compartment. Abstract entities show their name in italics. The core building block.

Enumeration — A box stereotyped «enumeration» listing a fixed set of allowed values (e.g. a status set).

Package — A resizable container that groups related entities. A package can be **collapsed** into a drill-down folder that links to a child diagram, keeping large models navigable.

Note — A folded-corner box holding free-text commentary, tied to what it explains by a dashed anchor.

RELATIONSHIPS

Association — A plain line for a general relationship; may carry a name and reading direction. **Aggregation** — Hollow diamond: a “has-a” whole/part where parts can exist independently. **Composition** — Filled diamond: a strong whole/part where parts die with the whole. **Generalisation** — Hollow triangle: an “is-a” inheritance from child to parent. **Realisation** — Dashed line + hollow triangle: a class implements an interface/contract.

Dependency — Dashed line + open arrow: one element uses another. **Containment** — Circle-plus ($\oplus$): an element belongs inside a package.

END DECORATIONS

Each association end can show a **multiplicity** (1, 0..*, 1..*), a **role name** with **visibility** (+ public, - private, # protected, ~ package — roles default to +), a **navigability** arrow, and a **constraint** in braces ({ordered}, {unique}, {readOnly}, {union}, or free text). Constraints sit outermost, beyond the role and multiplicity labels.

Tip: Multiplicities and roles stay anchored to the connector end; constraint boxes are nudged clear so nothing overlaps.